```
// ======================================================
// BENCHMARK CODE - MODE A (SOFTWARE POLLING LATENCY)
// ======================================================

const int triggerPin = 4;   // Virtual Tap Generator
const int interruptPin = 2; // Software Polling Input Pin

void setup() {
  Serial.begin(9600);

  pinMode(triggerPin, OUTPUT);
  digitalWrite(triggerPin, HIGH); // Default HIGH (active-LOW logic)

  pinMode(interruptPin, INPUT_PULLUP);

  delay(1000);
  Serial.println("========================================");
  Serial.println("   STARTING MODE A (POLLING) BENCHMARK    ");
  Serial.println("========================================");

  unsigned long totalElapsedSum = 0;

  for (int i = 0; i < 15; i++) {
    // 1. Record start timestamp right before pulse
    unsigned long t0 = micros();

    // 2. Start active-LOW pulse
    digitalWrite(triggerPin, LOW);

    // 3. Poll repeatedly until pin goes LOW
    while (digitalRead(interruptPin) == HIGH) {
      // Polling loop execution delay
    }

    // 4. Record detection timestamp immediately upon loop exit
    unsigned long t1 = micros();

    // Complete the 500us pulse width and restore HIGH state
    delayMicroseconds(500);
    digitalWrite(triggerPin, HIGH);

    // 5. Calculate per-trial latency (t1 - t0)
    unsigned long elapsed = t1 - t0;
    totalElapsedSum += elapsed;
```

```
    // 6. Print Trial Result
    Serial.print("Trial ");
    Serial.print(i + 1);
    Serial.print(": ");
    Serial.print(elapsed);
    Serial.println(" microseconds");

    // 7. Wait 200ms before next trial
    delay(200);
  }

  // 8. Calculate and print average
  unsigned long averageLatency = totalElapsedSum / 15;

  Serial.println("----------------------------------------");
  Serial.print("MODE A AVERAGE LATENCY: ");
  Serial.print(averageLatency);
  Serial.println(" microseconds");
  Serial.println("========================================");
}

void loop() {
  // Benchmark runs once in setup
}
```

```
// ======================================================
// BENCHMARK CODE - MODE B (HARDWARE INTERRUPT LATENCY)
// ======================================================

const int triggerPin = 4;   // Virtual Tap Generator
const int interruptPin = 2; // Hardware Interrupt Input Pin

volatile unsigned long t1 = 0;
volatile bool interruptTriggered = false;

void setup() {
  Serial.begin(9600);

  pinMode(triggerPin, OUTPUT);
  digitalWrite(triggerPin, HIGH); // Default HIGH (active-LOW logic)

  pinMode(interruptPin, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(interruptPin), sensorISR, FALLING);

  delay(1000);
  Serial.println("==========================================");
  Serial.println("  STARTING MODE B (INTERRUPT) BENCHMARK   ");
  Serial.println("==========================================");

  unsigned long totalElapsedSum = 0;

  for (int i = 0; i < 15; i++) {
    interruptTriggered = false; // Reset ISR flag

    // 1. Record start timestamp right before pulse
    unsigned long t0 = micros();

    // 2. Send 500us active-LOW pulse
    digitalWrite(triggerPin, LOW);
    delayMicroseconds(500);
    digitalWrite(triggerPin, HIGH);

    // 3. Wait for ISR to record t1 and set flag
    while (!interruptTriggered) {
      // Waiting for ISR execution
    }

    // 4. Calculate per-trial latency (t1 - t0)
    unsigned long elapsed = t1 - t0;
```

```
    totalElapsedSum += elapsed;

    // 5. Print Trial Result
    Serial.print("Trial ");
    Serial.print(i + 1);
    Serial.print(": ");
    Serial.print(elapsed);
    Serial.println(" microseconds");

    // 6. Wait 200ms before next trial
    delay(200);
  }

  // 7. Calculate and print average
  unsigned long averageLatency = totalElapsedSum / 15;

  Serial.println("----------------------------------------");
  Serial.print("MODE B AVERAGE LATENCY: ");
  Serial.print(averageLatency);
  Serial.println(" microseconds");
  Serial.println("========================================");
}

void loop() {
  // Benchmark runs once in setup
}

// INTERRUPT SERVICE ROUTINE
void sensorISR() {
  t1 = micros(); // Capture timestamp at hardware level
  interruptTriggered = true;
}
```